



Instituto Infnet

Do meu computador para a internet

Projeto de Bloco: Inteligência Artificial Aplicada

Aula 5 · Etapa 3 · Ambiente e publicação

Prof. Renan Santos

Onde paramos (Aula 2)

O painel coleta dados de verdade — e roda perfeitamente... no seu laptop.

- **Coleta real via API do IBGE**

27 UFs, ao vivo, com requests + json.

Sem pandas: lista de dicionários.

- **Visualização e robustez**

Filtro por região, métricas, gráfico.

@st.cache_data + fallback para o cache local.

- **Uma condição escondida no 'funciona'**

Funciona em QUEM tem Python, as bibliotecas certas, nas versões certas.

É essa condição que cai hoje.

► A Competência 1 terminou em 'eu vejo os dados'. Hoje começa 'outra pessoa vê'.



O e-mail que muda tudo

A gestora da ONG viu o painel na reunião — e pediu o link.

- **"Pode mandar o link para o pessoal da secretaria?"**

Não existe link. Existe uma pasta e um comando.

`streamlit run app.py` só funciona para quem tem o ambiente.

- **Ela não vai instalar Python**

Nem deveria: quem usa um painel não deveria precisar montá-lo.

O atrito está do NOSSO lado.

- **A pergunta da aula**

O que falta entre 'roda aqui' e 'qualquer pessoa abre no navegador'?

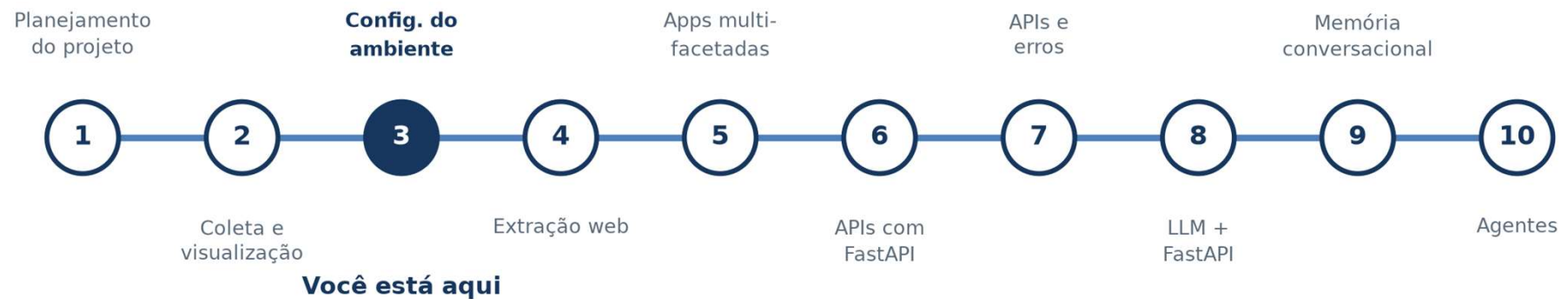
▶ **Sem deploy, o trabalho existe — mas só para você.**

'roda aqui' → ??? → 'qualquer pessoa abre'

(espaço para anotações na lousa)

Onde estamos no Projeto de Bloco

Etapa 3: configuração do ambiente. Começa a Competência 2 — e o TP2.



Objetivos da aula

Cinco capacidades — tudo o que separa o laptop da internet.

- **Montar um ambiente reprodutível**
venv + requirements.txt + .gitignore.
- **Preparar o projeto para deploy**
Caminhos robustos, dependências declaradas.
- **Publicar o repositório no GitHub**
init → add → commit → remote → push.
- **Implantar no Streamlit Community Cloud**
E atualizar com um novo push.
- **Separar segredo de código e ler logs**
st.secrets; diagnosticar quando quebrar.

O entregável muda: TP2

A Competência 2 não aceita print de tela — ela pede artefatos vivos.

- **TP1 (Comp. 1): um PDF do projeto**

A prova de que funciona era a sua palavra + um print.

- **TP2 (Comp. 2): PDF + repositório + app no ar**

A prova de que funciona é qualquer pessoa abrir a URL.

Abre nesta Etapa 3 e fecha na Etapa 4 (WebScraping).

- **Ao fim de hoje você tem um link**

Para mandar para a gestora — e para colocar no seu portfólio.

► **Publicar deixou de ser opcional: virou parte do entregável.**



1 · Ambiente reprodutível

Um programa não é só o seu código

código + Python + bibliotecas + versões + dados + configuração

- **Quando você diz 'funciona'...**

...você diz: funciona NESTA combinação.

Mude qualquer peça e a frase pode virar mentira.

- **Reprodutibilidade**

Qualquer pessoa recria a combinação a partir do que está DECLARADO no repositório.

Inclusive você, em outra máquina — e o servidor do Community Cloud.

- **O teste que resolve a dúvida**

Se eu apagar a sua .venv agora, você reconstrói? com qual arquivo?

► 'Funciona na minha máquina' não é desculpa: é o nome de um defeito.

O que vai (e o que não vai) para o repositório

O ambiente não é COPIADO — é RECONSTRUÍDO a partir da receita.

O ambiente não é copiado — é reconstruído



Versiona-se o que o app **PRECISA** para funcionar. Não se versiona o que é **reconstruível** (.venv/) nem o que é **secreto** (secrets.toml).

As três peças do ambiente

Cada uma responde a uma pergunta diferente — e a falta de qualquer uma quebra o deploy.

- **venv — ONDE as bibliotecas moram**

`python -m venv .venv`. Isola este projeto dos outros.

NÃO vai para o Git: é reconstruível, e é compilado para UM sistema operacional.

- **requirements.txt — QUAIS bibliotecas, em que versão**

É a receita. VAI para o Git.

Sem ele o servidor não instala nada → `ModuleNotFoundError` [Richards, p183].

- **.gitignore — o que NÃO entra**

`.venv/`, `__pycache__/` (peso) e `secrets.toml`, `.env` (segurança).

Precisa existir ANTES do primeiro `git add`.

► **Versiona-se o necessário para funcionar; nunca o reconstruível nem o secreto.**

A receita do ambiente

requirements.txt

```
streamlit>=1.40
```

```
requests>=2.28
```

```
urllib3>=2.0
```

```
# urllib3 já vem junto com requests – MAS nós o importamos
```

```
# diretamente em src/data_access.py. O que se importa, se declara.
```

```
# Regra: instalou biblioteca nova -> acrescenta AQUI na mesma hora.
```

À mão ou automático? (pipreqs)

O livro recomenda gerar o requirements.txt automaticamente. Nós escrevemos à mão — de propósito.

- **O que o livro faz**

pip install pipreqs; pipreqs . — lê os imports e escreve o arquivo [Richards, p183–184].
Ótimo para projetos grandes, onde ninguém lembra de tudo.

- **Por que aqui escrevemos à mão**

O pipreqs vê imports; dependências INDIRETAS podem escapar.
E ele grava as versões DA SUA MÁQUINA, que podem ser mais novas que o necessário.

- **Com 3 dependências, à mão é mais honesto**

Você entende cada linha — e é isso que o TP2 vai cobrar.

► Automatizar é bom quando você já sabe o que a automação faz por você.

O que nunca entra no repositório

```
.gitignore
```

```
# peso: reconstruível / gerado
```

```
.venv/
```

```
__pycache__/
```

```
*.py[cod]
```

```
# SEGURANÇA: o repositório do Community Cloud é PÚBLICO por padrão
```

```
.streamlit/secrets.toml
```

```
.env
```

Caminhos que sobrevivem à mudança de máquina

src/data_access.py

frágil: resolvido a partir de ONDE você rodou o comando

CACHE = "data/sample/ufs_cache.json"

robusto: ancorado no PRÓPRIO arquivo .py

BASE = Path(__file__).resolve().parent.parent

CACHE = BASE / "data" / "sample" / "ufs_cache.json"



2 · Do projeto ao repositório

Recap de 60 segundos: Git ≠ GitHub

Retomando o mini-deck da Aula 2 — agora aplicado ao projeto de verdade.

- **Git = versionamento NA SUA MÁQUINA**

GitHub = hospedagem NA NUVEM.

- **Nada sincroniza sozinho**

push é você quem dá. salvar ≠ commit ≠ push.

- **O que basta saber hoje**

'criar repositórios e atualizá-los conforme atualizamos a aplicação' [Richards, p174].

► **O repositório deixa de ser exercício e vira a fonte do que está publicado.**

Instalar e assinar seus commits

terminal

```
# Windows: https://git-scm.com/download/win    (ou: winget install Git.Git)
# macOS:   brew install git                      [Richards, p175]
# Linux:   sudo apt install git

git config --global user.name "Seu Nome"
git config --global user.email seu@email
```

O primeiro repositório do projeto

terminal

```
cd demo/painel_ods_brasil
git init -b main          # 'main', não 'master'

# >>> criar o .gitignore ANTES do primeiro add <<<
git status                # confira o que vai entrar
git add .
git commit -m "Painel ODS v3: pronto para publicar"
```

Conectar ao GitHub e enviar

terminal

```
# no site: New repository -> nome -> Create repository [Richards, p180-181]
```

```
git remote add origin https://github.com/SEU_USUARIO/painel-ods-brasil  
git push -u origin main
```

```
# origin = apelido da URL remota
```

```
# -u grava o vínculo: dali em diante basta 'git push'
```

O ciclo, dali em diante

add → commit → push. Três comandos que você vai repetir pelo resto do bloco.

- **git add .**

Escolhe o que entra na próxima versão.

- **git commit -m "..."**

Salva a versão. A mensagem diz O QUE mudou e por quê.

'use git add (...), git commit -m (...), and then finally, git push' [Richards, p183].

- **ANTI-PADRÃO: um commit gigante 'subi tudo'**

O histórico existe para você poder VOLTAR.

Commit grande demais não te deixa voltar a lugar nenhum útil.

► **Commits pequenos e descritivos são um favor que você faz a si mesmo no futuro.**



3 · Publicar no Streamlit Community Cloud

Três formas de publicar

Escolha consciente — e a nossa é a gratuita e mais simples.

- **Streamlit Community Cloud — grátis**

'easiest and preferred method for most Streamlit users' [Richards, p172].

Nossa escolha: protótipo, portfólio, TP.

- **AWS / Heroku — pago**

Controle de recursos, requisitos corporativos.

Cap. 8 do livro; fora do escopo do bloco.

- **Hugging Face Spaces — grátis**

Forte para apps de modelos/ML. Alternativa real.

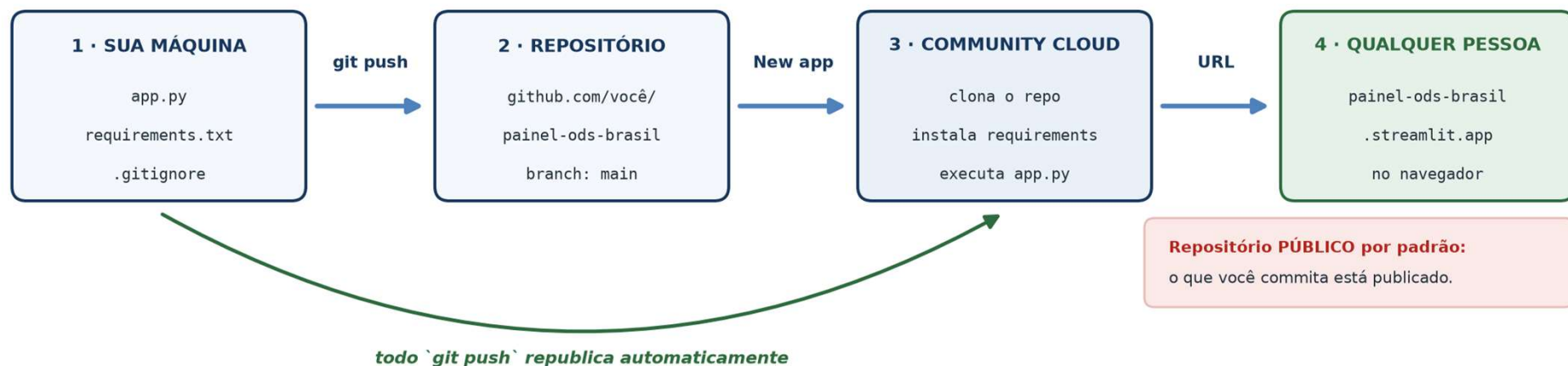
Volta a fazer sentido nas Etapas 8–10 (LLMs).

► **Grátis, com 1 GB de RAM por app [Richards, p267] — de sobra para o Painel ODS.**

O contrato de quatro itens

'Streamlit Community Cloud runs using GitHub' [Richards, p174] — por isso o §2 veio antes.

Do laptop à URL pública — e o contrato de quatro itens



Toda falha de deploy é a falta de um dos quatro: repositório • branch • arquivo principal • requirements.txt.

O fluxo, passo a passo

Não decore botões — a interface muda. Entenda o que o serviço precisa saber.

- **1 · share.streamlit.io, entrar com o GitHub**
Cadastro gratuito em share.streamlit.io/signup [Richards, p173].
- **2 · New app → repositório · branch · arquivo principal**
No nosso caso: `painel-ods-brasil · main · app.py` [Richards, p185].
- **3 · Deploy**
Ele clona o repo, instala o `requirements.txt` e executa o `app.py`.
Devolve uma URL própria — a sua.

▶ Quatro perguntas, quatro respostas. Se ele reclamar, falta uma delas.

O hábito novo: git push é publicar

'Whenever we make changes to the GitHub repository, we will see such changes reflected in the app.' [Richards, p185]

- **Mude o título, salve, e: add → commit → push**

Volte à URL e recarregue.

'It may take a couple of minutes for the app to reload!' [Richards, p187]

- **O rodapé do app mostra a versão publicada**

É como você confirma, olhando a página, que o push chegou lá.

- **A consequência que muda tudo**

O repositório é a FONTE DA VERDADE do que está no ar.

Se não está commitado, não está no ar.

► Não existe 'subir o arquivo no servidor'. Existe commitar e empurrar.

Quando quebra: leia o log antes de mexer

Manage app → logs; dali dá para reiniciar, excluir e baixar os logs [Richards, p188].

- **ModuleNotFoundError: No module named 'X'**
Dependência não declarada → acrescente ao requirements.txt e dê push.
- **FileNotFoundError:json**
Caminho relativo ao diretório de execução → ancore em Path(__file__).
- **Error installing requirements**
Versão inexistente ou erro de digitação → corrija a linha e dê push.
- **Regra de ouro do diagnóstico**
Leia a ÚLTIMA linha do traceback antes de mudar qualquer coisa.

► A resposta está no log, não no palpite. [Richards, p80: depure com `st.write()`]

Segredo fora do código: st.secrets

app.py

```
# ❌ SENHA = "minha-senha"      # vai para o repositório PÚBLICO

# ✅ o segredo mora fora do código:
if st.text_input("Senha", type="password") != st.secrets["senha"]:
    st.stop()                  # interrompe o script aqui

# local: .streamlit/secrets.toml (no .gitignore!)
# no ar: Edit secrets, no painel do Community Cloud
```

Segredo commitado não se apaga

'the default in Streamlit Community Cloud is to use public GitHub repositories with entirely public code, data, and models.'
[Richards, p188]

- **Tudo que você commita está publicado**
Não é 'ficará exposto se alguém procurar'. Está exposto agora.
- **Apagar no commit seguinte NÃO resolve**
O histórico do Git guarda a versão anterior — é o trabalho dele.
A chave precisa ser REVOGADA e trocada.
- **Por que aprender isso hoje, sem chave nenhuma**
A API do IBGE é aberta: dá para errar sem custo.
No TP2 e nas Etapas de LLM, o erro custa dinheiro.

► **Trate tudo que você commita como publicado — porque está.**

Temos o painel no ar!

Mesma tela da Aula 2 — mas agora com uma URL que qualquer pessoa abre.

O resultado da Aula 5: um link que qualquer pessoa abre



Nada mudou na tela — mudou QUEM consegue chegar até ela.

Princípios que levamos

Quatro ideias que valem para qualquer projeto que precise sair da sua máquina.

- **Reprodutibilidade**

O ambiente é DECLARADO, não copiado.
Se não está declarado, não existe.

- **Paridade dev/prod**

O que roda aqui tem de rodar lá.
Toda diferença vira configuração explícita — nunca linha escondida.

- **Segredo ≠ código**

Repositório público é o padrão; o histórico não esquece.

- **Publicar cedo e frequentemente**

App imperfeito no ar recebe feedback; app perfeito no laptop não recebe nada.

► **Publicar é engenharia — e é o que transforma análise em produto.**



O caso, em 3 lições (v3)

O que a preparação DESTA material revelou — tropeços reais, não hipotéticos.

- **O verify=False que salvou a Aula 2 virou risco**
Workaround de rede escolar, versionado em repositório público.
Virou configuração com padrão seguro. Todo workaround tem prazo de validade.
- **requirements.txt incompleto derrubou o primeiro deploy**
requests estava instalado localmente e ausente do arquivo.
É a definição operacional de 'funciona na minha máquina'.
- **O caminho relativo só quebrou DEPOIS de publicar**
O fallback falhava justamente quando mais importava.

► **Método + ambiente declarado + segredo fora do código = um app que você publica sem medo.**

Para a próxima aula

O painel está no ar — mas quem abre o link não consegue explorar.

- **Interação dinâmica (Aula 6)**

Botões, seletores, slider, formulário e memória de sessão.

- **E o melhor: cada melhoria vai ao ar na frente de vocês**

git push → o app publicado muda. Sem cerimônia.

- **Seu TP2 agora**

Publique o SEU projeto e traga a URL na próxima aula.

► O caso continua: v4 = o painel que responde ao usuário.

Em 3 frases

O mapa mental da Aula 5.

- **Declare o ambiente**

venv + requirements.txt + .gitignore — a receita, não a cópia.

- **Publique pelo repositório**

GitHub é a porta de entrada do Community Cloud.

- **Empurre para publicar**

git push atualiza o app; segredo nunca vai junto.

► **Do laptop (Aula 2) à URL pública (hoje).**